

# Project Proposal: Regional Nectar-Flow Forecast

10-Day Hive-Weight Prediction for the Central-Swiss Pre-Alps

I.BA\_MLOPS · HS26

Elias Christen

<https://github.com/Whalesharkking/mlops>

## 1. Problem statement

The goal is to predict how much weight the monitored bee colonies in the *Nordalpen* region gain or lose each day, for today and the following nine days. Nordalpen is a Central-Swiss pre-alpine zone covering roughly the cantons of Lucerne, Schwyz, Uri and Ob-/Nidwalden. The forecast is a series of 10 daily values (horizon  $h = 1 \dots 10$  days) and is refreshed every morning, once the nightly scale upload and the new weather forecast are in.

The daily weight change is measured in kilograms, after beekeeper actions (feeding, honey removal, adding boxes) are filtered out. A positive value means the colonies collect more nectar than they eat, a negative value means they live off their stores. It is averaged over the roughly 372 scales in the region, so the forecast describes the regional nectar flow, not a single hive. This helps beekeepers decide when to harvest, when to feed before a treatment, and when a nectar gap (“Trachtlucke”) or a strong flow is coming.

Success criterion (MAE in kg/day, reported per horizon): the model is compared with two simple baselines. *Persistence* assumes every coming day repeats the last complete day. *Climatology* predicts the usual change for that date, averaged over 2019 to 2023 ( $\pm 7$  days). On the 2026 data persistence wins for today (MAE 0.24 vs. 0.32) but falls to 0.45 at day 10, while climatology stays at 0.32. The model must therefore beat the better of the two by at least 10 % for  $h = 1 \dots 5$  and at least match it for  $h = 6 \dots 10$ .

## 2. Originality & motivation

I am an active beekeeper, so I know the problem first-hand and can judge whether the output is useful in the apiary. I checked the HSLU projects on [mlops-lab.ch](https://mlops-lab.ch) and the KTH ID2223 lists. The closest are a pollen-concentration forecast and a bark-beetle outbreak predictor, and none uses hive-scale data. What makes it different is the data: a Swiss citizen-science network of hundreds of connected scales, combined with archived weather forecasts. They predict nectar income, which depends on the weather but cannot be read from it. The real question is whether a weather forecast lets the model beat what a beekeeper already knows from experience, which is exactly what the climatology baseline captures.

## 3. Data source & features

**Hive data.** The data comes from the HiveWatch / BienenSchweiz network behind the public scale map on [bienen.ch](https://bienen.ch). Its JSON endpoint `region/11/averages` returns the intervention-filtered weight series (`ext_weight`) of the region, hourly since 2019 (about 65'000 points today), growing by one day every night. It needs no key, is polled once a day and credited. Fallbacks: the network's own daily-delta endpoint (`midnightvalues`) as a second access path, and the open Hiveeyes scale network as an alternative provider.

**Weather data.** The forecasts come from the Open-Meteo Single Runs API. It keeps every past run of the ECMWF IFS HRES model (10 days ahead) since 14 March 2024, about 925 runs so far. Each morning the pipeline takes the 00 UTC run for four towns (Lucerne, Schwyz, Altdorf, Sarnen), averages them and turns the hourly values into daily values. Each row is stored with two dates: the day the forecast was made and the day it is for. So the model always learns from real forecasts, never from the weather that actually happened, and the backfill uses the same code as the daily run. Fallback: the Open-Meteo Previous Runs API (same model, but only 7 days ahead, climatology after that).

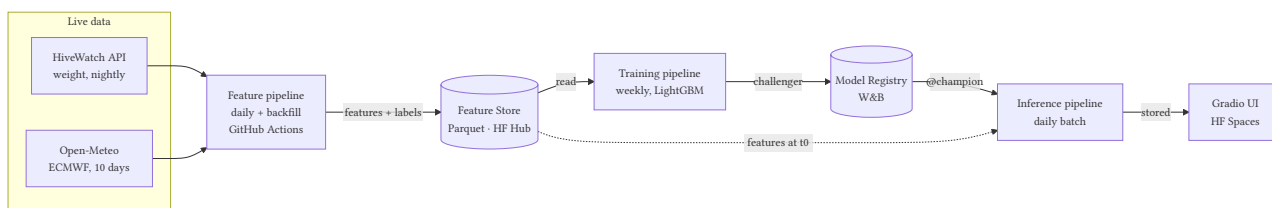
**Label.** The net weight change of one day: the weight at midnight after the day minus the weight at midnight before it. At midnight all bees are home, and the result matches the daily changes HiveWatch publishes itself (within 0.004 kg). Days without a midnight value stay empty and are not interpolated. Because scales can upload late, a label counts as final only after two days. The label comes from the scale data alone and cannot be derived from any feature.

**Features.** There is one row per forecast day  $t_0$  and horizon  $h$ , and all features are batch features. The weather forecast for the target day gives max and min temperature, precipitation, shortwave radiation and max wind. The weight momentum is the net change over the last 1, 3, 7 and 14 days before  $t_0$ , on a fixed daily grid so a 1-day lag is always a real day. The climatology value of the target day brings the 2019 to 2023 history into the model. The last features are the day of year (as sine and cosine) and  $h$  itself.

**Split and leakage.** Training uses forecast days from 14 March 2024 to 22 December 2025, so no training target falls into 2026. The test set is the unseen 2026 season (1 January to today), reported overall, per  $h$  and for the flow season April to July. The climatology uses only 2019 to 2023, so it contains no training or test year. This is a regression task, so there is no rare class.

## 4. System design

All three pipelines are batch jobs. The model predicts in advance (offline or batch prediction), and the UI only shows the stored forecasts (request-response). Streaming would add no value, because the data arrives once a night.



### 4.1. Core

The **feature pipeline** runs every morning after the 00 UTC weather run is out and writes features and labels to the feature store. A backfill run fills the history from March 2024 with the same code. The **training pipeline** retrains a LightGBM model (fast on small tabular data, no GPU) from scratch once a week on all rows with a final label (automated stateless retraining on natural labels). Each new model is registered as a challenger with its Git commit and dataset version. It becomes champion only if it beats the current champion and both baselines on the last eight weeks of final labels, which it did not see in training. The **inference pipeline** loads @champion and stores the 10-day forecast next to the baselines for the UI.

### 4.2. Tech stack

**HuggingFace Hub** versioned Parquet as a “poor man’s feature store”, free and next to the UI

**Weights & Biases** experiment tracking and model registry with aliases, free cloud, no server to run

**GitHub Actions** free cron scheduler next to the code, runs the daily, weekly and backfill jobs

**HuggingFace Spaces (Gradio)** serves the forecast and the model version, the required cloud deployment

**Docker with pinned dependencies** reproducible runs locally, in CI and on the Space

Only two secrets are needed (HuggingFace and W&B tokens), kept in GitHub Secrets. The repository is public.

### 4.3. Optional

Only if time allows: a monitoring job that computes the live MAE from stored forecasts and late labels, a drift check on weight and weather, retraining when the live MAE rises, a 16-day horizon from the 2027 season (GFS runs), and a mirror on GCP Cloud Run.